Training at Scale

Lesson 5 of 12 · Where the memory goes, and how to split a model across GPUs.

In this lesson → ① The 16-byte ledger ② Sharding ③ Recompute ④ GPU-hours ⑤ Splitting a model ⑥ The bubble ⑦ LoRA
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

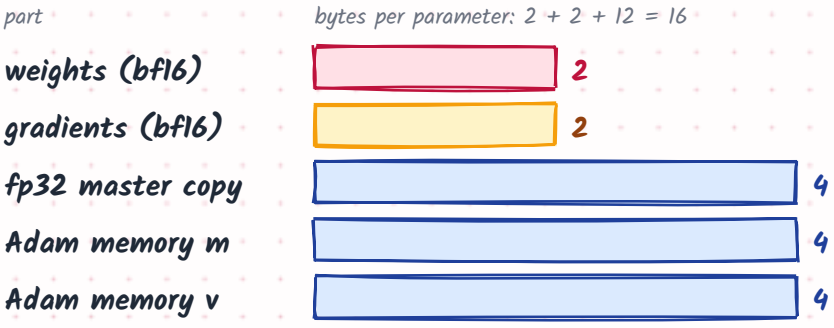
1 Training in one picture

- **Forward**: the model guesses
- **Loss**: how wrong was it?
- **Backward**: who is to blame? (the **gradients**)
- **Optimiser**: nudge every weight, repeat billions of times

2 Four things in memory

Part	Scales with
weights	parameters
gradients (one per weight)	parameters
optimiser state (Adam keeps two numbers per weight, plus a master copy)	parameters
activations (saved mid-results for the backward pass)	tokens in flight

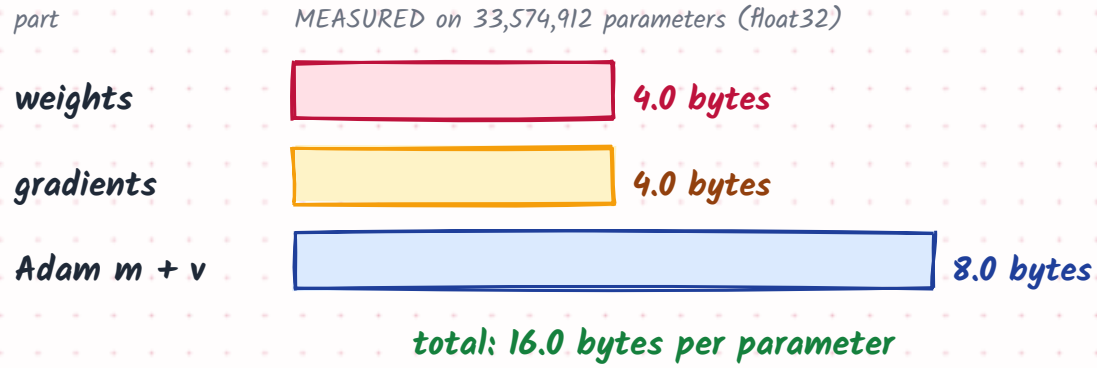
3 The 16-byte ledger



> **training state ≈ (2 + 2 + 12) × parameters = 16 bytes per parameter** (the weights are only an eighth)

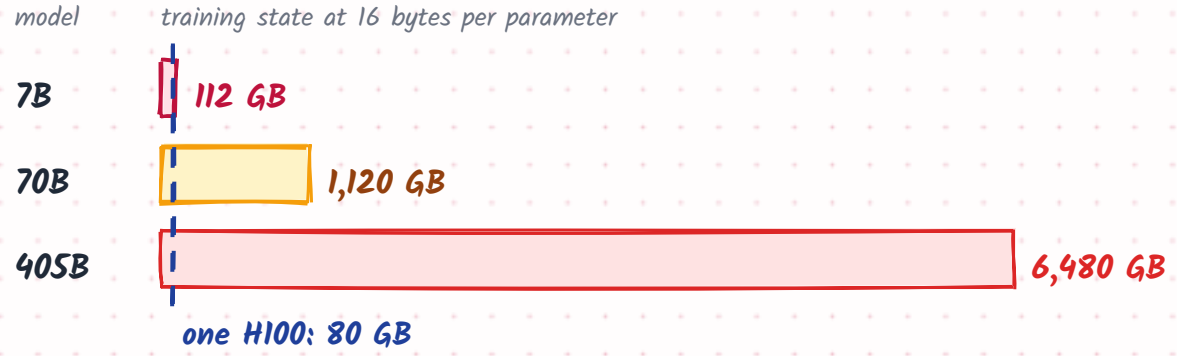
CLOUD NOTEBOOK

4 Measured on a real model



33,574,912 parameters, one Adam step, float32 on a CPU: 4.0 + 4.0 + 8.0 = **16.0** bytes per parameter. Different recipe, same total. Script: l05_memory_ledger.py.

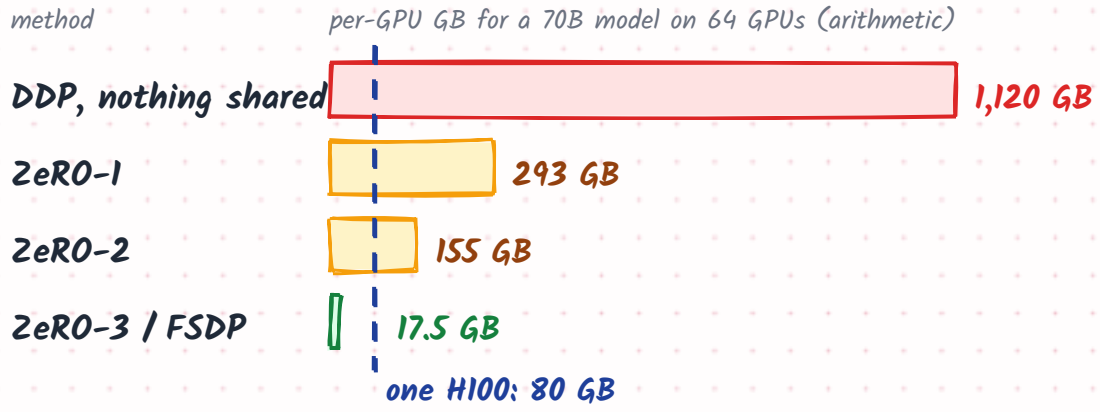
5 Scaling up



Model	Training state
7B	112 GB
70B	**1,120 GB** (weights alone: 140 GB)
405B	6,480 GB

Arithmetic. Activations come on top.

6 Sharding: ZeRO and FSDP



Method	Shards	7B, 8 GPUs	70B, 64 GPUs
DDP	nothing	112 GB	1,120 GB
ZeRO-1	optimiser state	38.5 GB	293 GB
ZeRO-2	+ gradients	26.2 GB	155 GB
ZeRO-3 / FSDP	+ parameters	14.0 GB	**17.5 GB**

Why stage 3
ZeRO-1/2 still keep a full bf16 copy of the weights on every GPU (140 GB for 70B). Only ZeRO-3 / FSDP fits. Price: about 1.5x the network traffic of DDP.

7 Activations and recomputation

> Activations are the mid-results saved in the forward pass. They grow with tokens, and with attention they can grow with sequence length squared. FlashAttention avoids the biggest piece. The standard trick for the rest: **recompute**.

Keep only each block's input. When the backward pass needs the inside of a block, re-run that block's forward pass.

activation memory

793 MB → 426 MB

time per step

5.6 s → 6.9 s

in plain words

half the memory, a fifth more time

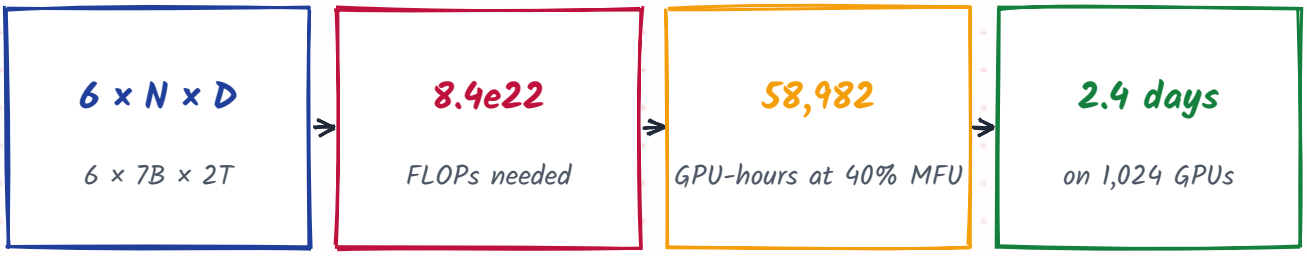
measured on a CPU, 12 blocks, 2,048 tokens

Setting	Seconds / step	Peak memory	Activation part
keep everything	5.55	1,721 MB	793 MB
recompute per block	6.90	1,354 MB	**426 MB** (0.54x)

MEASURED on one CPU thread, 12 blocks, 2,048 tokens, separate processes. Time cost 1.19x to 1.24x across runs; the notebook rule of thumb is up to +33%. Script: l05_recompute.py.

8 Counting compute

training FLOPs ≈ 6 × N × D (N parameters, D tokens): 2 forward + 4 backward per parameter per token.
GPU-hours = 6ND ÷ (peak × MFU × 3600)



at an assumed $2 per GPU-hour: about $118,000

Run	FLOPs	GPU-hours	At $2 (assumed)
1B on 0.1T tokens	6.0e20	421	$843
7B on 2T tokens	8.4e22	**58,982**	$117,964
70B on 1.4T tokens	5.9e23	412,875	$825,750

H100 dense BF16 peak 989 TFLOP/s (vendor), MFU 40% (notebook: well-tuned dense runs reach about 35 to 45%). Script: l05_flops_math.py.

9 MFU honestly

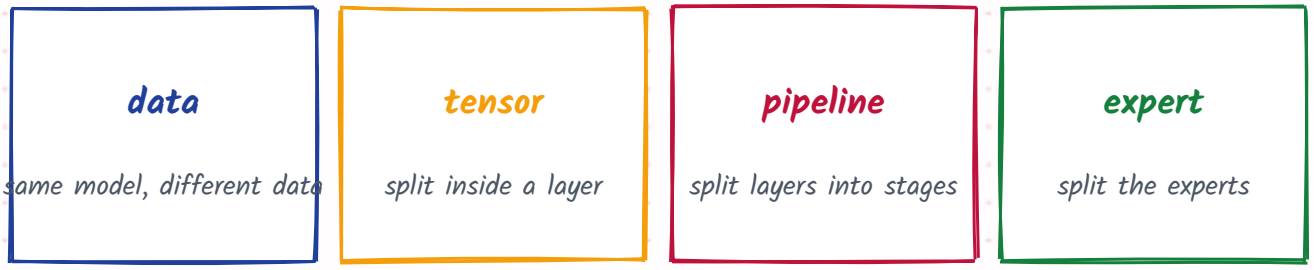
- ✓ MFU = needed FLOPs per second ÷ hardware peak. Name the peak: FP8 runs divide by a bigger peak, so they are not comparable with BF16 MFU
- ✓ Quote **MFU**, not HFU: HFU also counts recomputed work
- ✓ DeepSeek-V3 reverse check: about 33% against the BF16 peak, about 17% against the FP8 peak

10 How many tokens?

Model	Tokens per parameter
Chinchilla-optimal (2022)	about 20
Llama 3 405B (reported)	about 38
7B on 2T tokens	about 286

Modern models over-train on purpose: a smaller, longer-trained model is cheaper to run afterwards.

11 Four ways to split

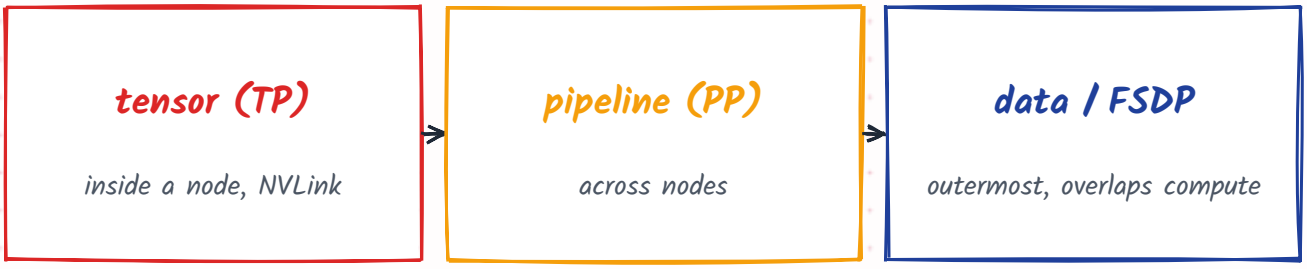


each one cuts a different thing

12 Who splits what

Strategy	Splits	Traffic	Place on
Data / FSDP	the batch (and state)	gradients, parameters; overlaps compute	outermost
Tensor (TP)	matrices inside a layer	all-reduces every layer, on the critical path	**NVLink only**
Pipeline (PP)	layers into stages	boundary activations	across nodes
Expert (EP)	MoE experts	all-to-all, twice per MoE layer	NVLink domain

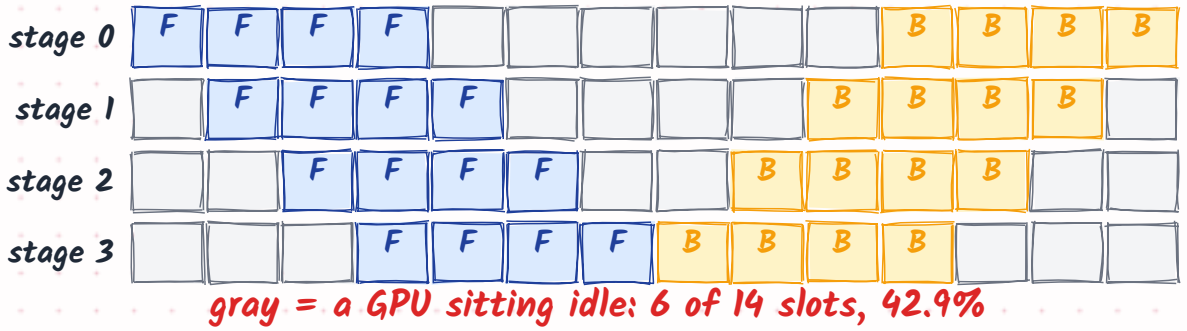
13 Mapping onto the hardware



the chattiest group sits closest

The more often a group talks, the faster the link it needs. Order, fastest link outwards: **T**ensor, **C**ontext, **E**xpert, **P**ipeline, **D**ata ("Tiny Cats Eat Pizza Daily"). Llama 3 405B (reported): TP 8 in-node, PP 16, DP up to 128.

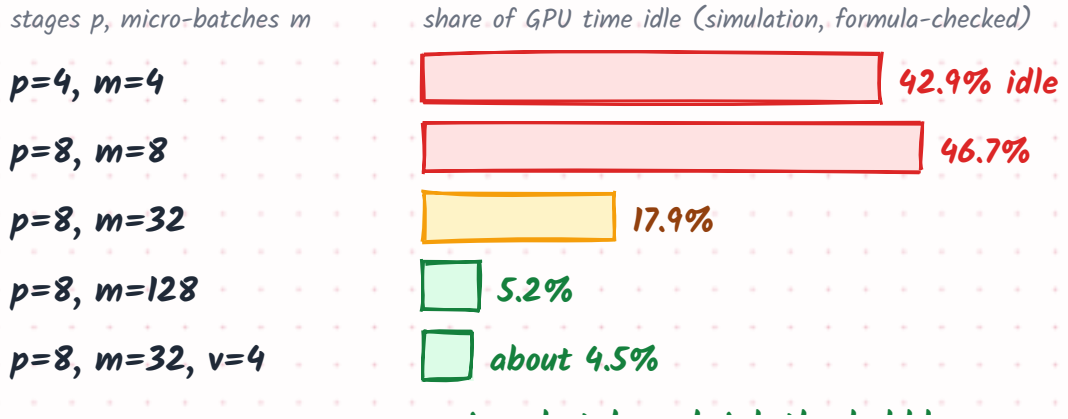
14 The pipeline bubble



> **bubble = (p − 1) ÷ (m + p − 1)** *p stages, m micro-batches per step*

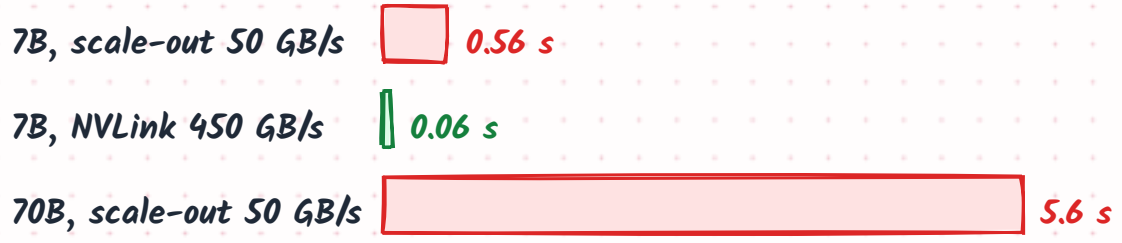
SIMULATION: GPipe schedule, forward and backward one slot each (real backward is about 2x). Counted idle slots match the formula exactly. Script: l05_bubble_sim.py.

15 Shrinking the bubble



p	m	Idle (simulated)	Interleaved x4 (formula)
4	4	42.9%	10.7%
8	8	46.7%	11.7%
8	32	17.9%	4.5%
8	128	5.2%	1.3%
16	128	10.5%	2.6%

16 The network cost



17 The silent slowdown

> The library that does group communications is **NCCL**. If the right network plugin or settings are missing it quietly falls back to a slower transport: roughly **2 to 5 times** slower, no error. On every new cluster: `export NCCL_DEBUG=INFO` and read which transport it picked.

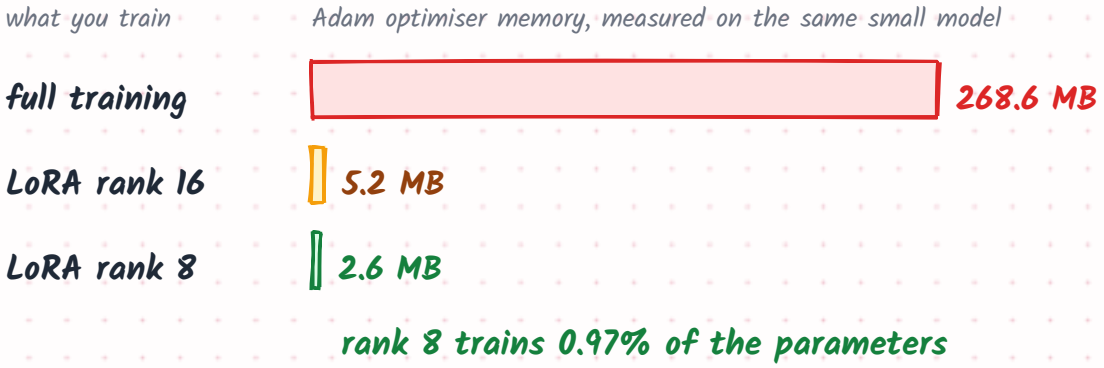
18 Number formats

Format	Status (notebook, Oct 2026)
BF16	the default
FP8	proven in production (DeepSeek-V3)
NVFP4	matched FP8 on a 12B model over 10T tokens (one report)

19 Failures are routine

Llama 3 reported **419 unexpected interruptions in 54 days**: 54 × 24 ÷ 419 = one every **3.1 hours**. Checkpoint on a schedule (lesson 2), save asynchronously (lesson 4), restart automatically.

20 Fine-tuning with LoRA



Setting	Trained parameters	Adam memory
full training	33,574,912 (100%)	268.6 MB
LoRA rank 8	327,680 (**0.97%**)	**2.6 MB**
LoRA rank 16	655,360 (1.91%)	5.2 MB

MEASURED on the same small model. Usual order: SFT, then DPO (preferences), then RL against a checker for some tasks. QLoRA also stores the frozen weights in 4 bits (described, not measured here).

21 Which tool?

Situation	Start with
custom model, pure PyTorch	FSDP2 / torchtitan
1,000+ NVIDIA GPUs, MoE	Megatron Core / Bridge
TPUs	JAX + MaxText
one machine to a few, fine-tuning	HF Trainer / Accelerate

22 ShopNest Copilot: fine-tune, do not pretrain

Plan	Memory
full fine-tune of 8B	8B × 16 bytes = **128 GB**
LoRA: frozen 8B in bf16	8B × 2 bytes = **16 GB** + small adapters, fits one card

23 Words to keep

Word	Meaning
Optimiser state	memory the optimiser keeps for each weight
Sharding	giving each GPU a slice instead of a copy
Recompute	extra compute for less memory
Bubble	idle time at the start and end of a pipeline

24 Interview corner

Q. How much memory does training need per parameter?

A. About 16 bytes with Adam in mixed precision: 2 + 2 + 12. I measured 16.0 on a real model.

Q. How do you fit a model that is too big for one GPU?

A. Shard the state (ZeRO-3 / FSDP), recompute activations, tensor parallel inside a server, pipeline across servers.

Q. What is the pipeline bubble?

A. Idle time at the start and end: (p-1)/(m+p-1). 4 stages and 4 micro-batches idle 42.9%; 8 stages and 128 micro-batches idle 5.2%.

Q. Why LoRA?

A. It trains under 1% of the parameters. On my small model the optimiser memory fell from 268.6 MB to 2.6 MB.

25 Your turn

105_memory_ledger.py, **105_recompute.py**, **105_bubble_sim.py**, **105_flops_math.py**: change the model size or micro-batches.

⚡ Revise in 60 seconds

- → training state = **16 bytes** per parameter
- → 70B: weights 140 GB, training **1,120 GB**
- → ZeRO-3 on 64 GPUs: **17.5 GB** each
- → recompute: **0.54x** activation memory, about 1.2x time
- → 6ND: 7B on 2T = **58,982** GPU-hours
- → bubble = (p-1)/(m+p-1)
- → chattiest split on the fastest link
- → LoRA rank 8: **0.97%** trained

Next lesson → CAI-06
Inference and serving: how to make a trained model answer fast

Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**